

AArch64 Data-Processing Instructions

Operand forms, bitwise operations, shifted registers,
and immediate constants in AArch64 assembly

Dr. J. Burns

Topic 5 Outline

Topic Objective

Use AArch64 data-processing instruction forms to combine register operands, immediate constants, bit masks, and shifted source values.

- **Section 1:** Data-processing instruction forms
- **Section 2:** Logical and bitwise instructions
- **Section 3:** Shifted register operands
- **Section 4:** Immediate encodings and constant construction
- **Section 5:** Summary and self-test

Section 1

Data-Processing Instruction Forms

Data Processing in Registers

From Topic 4 to Topic 5

Topic 4 covered what arithmetic results and shifts mean. Topic 5 focuses on how AArch64 expresses data-processing operations using registers, immediates, masks, and shifted operands.

`add x0, x1, x2` and `x3, x4, x5`

- The first operand is the destination register.
- The remaining operands provide the values used by the operation.
- The result replaces the previous contents of the destination register.
- Memory access remains separate: loads and stores move values between memory and registers.

Register and Immediate Arithmetic

Register Operand

add x0, x1, x2

sub x3, x4, x5

- Both source values come from registers.
- Useful when both values were computed earlier.

Immediate Operand

add x0, x1, #24

sub x3, x4, #8

- One source value is encoded in the instruction.
- Useful for small fixed constants.

32-Bit and 64-Bit Arithmetic

Register Width Selects the Operation Width

The same instruction family is available for 32-bit w registers and 64-bit x registers.

`add w0, w1, w2` `add x0, x1, x2`

- w0–w30 operate on 32-bit values.
- x0–x30 operate on 64-bit values.
- Writing a w register clears the upper 32 bits of the corresponding x register.
- Operand widths must match the instruction form.

Reusing a Source as the Destination

In-Place Update

The destination register may also be one of the source registers.

```
add x0, x0, #1
```

```
sub x5, x5, x2
```

- The sources are read before the destination value is written.
- `add x0, x0, #1` increments `x0`.
- `sub x5, x5, x2` replaces `x5` with `x5 - x2`.
- This form is common in counters and address calculations.

Arithmetic Forms Recap from Topic 4

Purpose of This Recap

These operations were introduced in Topic 4. Here they are placed in the AArch64 data-processing family so the later operand forms make sense.

Form	Example	Point for Topic 5
register arithmetic	<code>add x0, x1, x2</code>	both source values are registers
immediate arithmetic	<code>add x0, x1, #24</code>	one source value is encoded in the instruction
multiply	<code>mul x0, x1, x2</code>	register-only source operands
signed / unsigned divide	<code>sdiv / udiv</code>	signedness is part of the instruction choice

- The arithmetic meaning is review; the new focus is how operands are written and encoded.

Section 2

Logical and Bitwise Instructions

Bitwise Operations

Operate on Corresponding Bits

Logical instructions process each bit position independently. Signed and unsigned numeric interpretation is irrelevant to the bitwise operation itself.

`and x0, x1, x2` `orr x0, x1, x2` `eor x0, x1, x2`

- `and`: output bit is 1 only when both input bits are 1.
- `orr`: output bit is 1 when either input bit is 1.
- `eor`: output bit is 1 when the input bits differ.

AND for Selecting Bits

Masking

A bit mask uses 1s in positions that should be preserved and 0s in positions that should be cleared.

```
      11001010
    & 00111100
result 00001000
```

AArch64 Example

```
and x0, x1, x2
```

ORR for Setting Bits

Setting Selected Bits

A mask containing 1s can force selected result bits to 1 while leaving other positions unchanged.

```
      10000001
      | 00110000
result 10110001
```

AArch64 Example

```
orr x0, x1, x2
```

EOR for Toggling Bits

Toggle Selected Bits

XOR with 1 flips a bit; XOR with 0 leaves the bit unchanged.

```
      10100110
    ^ 00111100
result 10011010
```

AArch64 Example

```
eor x0, x1, x2
```

Inverted-Operand Logical Forms

Why These Instructions Exist

Some bit-mask operations need one operand inverted before AND, OR, or XOR. These forms combine the inversion with the logical operation.

Instruction	Operation	Typical use
bic	$A \wedge \sim B$	clear selected bits
orn	$A \vee \sim B$	OR with an inverted operand
eon	$A \oplus \sim B$	XOR with an inverted operand

Example: Clearing Selected Bits

```
bic x0, x1, x2
```

x2 is the mask: 1-bits in x2 clear the corresponding bits of x1; the result is written to x0.

Logical Instruction Patterns

Goal	Operation	Typical instruction
Keep selected bits	AND with 1s at wanted positions	<code>and</code>
Set selected bits	OR with 1s at wanted positions	<code>orr</code>
Toggle selected bits	XOR with 1s at wanted positions	<code>eor</code>
Clear selected bits	AND with inverted mask	<code>bic</code>

Bit Masks

Bit masks are common in device control, packed data, permission fields, status words, and instruction-field manipulation.

Section 3

Shifted Register Operands

Shifts: Brief Recall from Topic 4

Why This Is Only a Recall

Topic 4 covered the bit movement. This topic uses shift names as part of larger AArch64 data-processing instructions.

Shift	Bit movement	Typical meaning
lsl	left; zeros enter on right	scaling by powers of two when bits are not lost
lsr	right; zeros enter on left	unsigned right shift
asr	right; sign bit replicated	signed right shift
ror	outgoing bits re-enter at the other end	rotate a bit pattern

New Focus

AArch64 can attach a shift to a source register inside another operation.

Shifted Register Operands

Combine Shift and Operation

Many AArch64 data-processing register forms can shift one source operand as part of the same instruction.

```
add x0, x1, x2, lsl #3
```

```
orr x3, x4, x5, lsr #8
```

- The shift applies to the final source register before the main operation.
- The source register itself is not modified.
- This is not a separate shift instruction followed by a second operation.
- It combines scaling or bit positioning with the main operation in one instruction.

Section 4

Immediate Values and Constants

Immediate Operands Have Encoding Limits

Constant Inside the Instruction

An immediate operand is a constant represented inside the instruction rather than read from a source register.

```
add x0, x1, #5  
sub x2, x2, #32
```

- The # prefix identifies an immediate constant in AArch64 assembly syntax.
- Immediate forms are compact because no extra source register is needed.
- The constant must fit the immediate format for that instruction family.
- Arithmetic, logical, shift, and move-wide instructions do not all use the same immediate encoding.
- Exact bit fields are studied later; here the point is that immediate availability is instruction-specific.

Logical Immediate Masks

Useful but Restricted

Logical-immediate forms can encode many repeated bit-mask patterns directly, but not every arbitrary 32-bit or 64-bit constant.

```
and x0, x1, #0xff
orr x2, x2, #0x100
```

- These forms are especially useful for masking and setting fields.
- Whether a particular mask is encodable is determined by the logical-immediate encoding rules.
- If it is not encodable directly, the constant can first be constructed in a register.

Building Constants with `movz` and `movk`

`movz`

Moves a 16-bit immediate into a selected 16-bit position and sets the remaining register bits to zero. `movz x0, #0x1234`

`movk`

Moves a 16-bit immediate into a selected 16-bit position while keeping the other bits unchanged. `movk x0, #0xabcd, lsl #16`

Purpose

Several move-wide instructions can assemble a full 64-bit constant from 16-bit pieces without loading it from memory.

movn and mov

movn

- Moves the bitwise NOT of a 16-bit immediate into a selected position.
- Other bits are set consistently with the move-wide operation.
- Can efficiently construct constants containing many 1s.

mov

- Common assembler syntax for putting a value into a register.
- Depending on the operands, the assembler selects an appropriate underlying instruction form.
- It should not be assumed to correspond to one universal hardware encoding.

```
mov x0, x1    mov x2, #42
```

Constant Construction Example

Construct 0x123456789abcdef0

A 64-bit constant can be built as four 16-bit pieces.

```
movz x0, #0xdef0
movk x0, #0x9abc, lsl #16
movk x0, #0x5678, lsl #32
movk x0, #0x1234, lsl #48
```

Result

After the sequence, x0 contains 0x123456789abcdef0.

Choosing Between an Immediate and a Register

Use an Immediate When

- the constant is known at assembly time,
- it fits the instruction's immediate form, and
- no register value is required for reuse elsewhere.

Use a Register When

- the value is computed at runtime,
- the constant does not fit the immediate form, or
- the value will be reused by several instructions.

Compiler Role

Compilers and assemblers choose among these forms to express the required operation using legal A64 encodings.

Combined Data-Processing Example

Goal

Scale a value by 8, add it to a base value, then keep only the low byte.

```
add x0, x1, x2, lsl #3
and x0, x0, #0xff
```

- The first instruction shifts `x2` left by 3 and adds it to `x1`.
- The second instruction masks the result to retain only bits 7–0.
- Both operations remain entirely within the register data-processing model.

Section 5

Summary and Self-Test

Topic 5 Summary

- AArch64 data-processing instructions operate on register values, with some forms accepting immediate constants.
- w registers select 32-bit operations; writing a w register clears the upper 32 bits of the corresponding x register.
- and, orr, eor, and bic manipulate selected bit positions using masks.
- Shifted register operands combine a shift with another register operation.
- Immediate constants are limited by the encoding rules of each instruction family.
- movz, movk, and movn help construct constants that do not fit a simple immediate form.

Self-Test 1/2

- ① What is the difference between `add x0, x1, x2` and `add x0, x1, #8`?
- ② What happens to the upper 32 bits of `x0` after writing `w0`?
- ③ Why can the destination register also be one of the source registers?
- ④ Which logical instruction is useful for keeping only selected bits?
- ⑤ How can `eor` be used to toggle selected bits?

Self-Test 2/2

- 6 What does the shifted operand mean in `add x0, x1, x2, lsl #3`?
- 7 Why is a shifted operand not the same thing as modifying the source register?
- 8 Why can an immediate operand not represent every possible 64-bit constant?
- 9 What roles do `movz` and `movk` play when constructing a 64-bit constant?
- 10 Why might assembler syntax such as `mov x0, #42` not correspond to one single hardware encoding?

Data Transfer and Addressing

Topic 6 moves from register data processing to the AArch64 instructions that transfer values between registers and memory.

- Loads and stores
- Transfer widths and extension
- Base-plus-offset addressing
- Pre-indexed and post-indexed addressing
- Arrays, pointers, and structured memory access